# 在 Arch Linux 上用 qemu 运行 RISC-V Linux 虚拟机

## 实验环境

先写一下笔者使用的环境：系统是 Arch Linux (x86_64), 内核是 Linux-zen 6.18.9

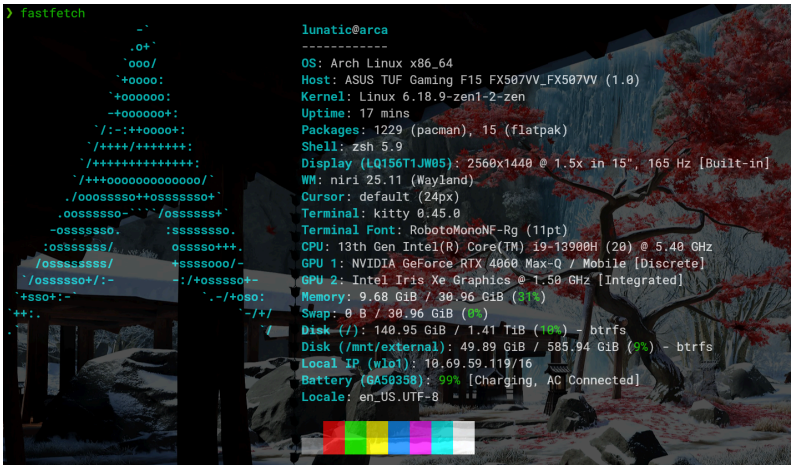


Figure 1: 实验环境

## 实验过程

### 1. 安装交叉编译工具链

我们需要安装 `riscv64-linux-gnu-gcc` 交叉编译工具链来编译 RISC-V Linux 内核和 BusyBox.

> **? `riscv64-linux-gnu-gcc` 还是 `riscv64-unknown-elf-gcc`**
>
> 他们的本质区别在于 `riscv64-linux-gnu-gcc` 是针对 Linux 系统的交叉编译工具链，适用于编译运行在 Linux 上的 RISC-V 程序；而 `riscv64-unknown-elf-gcc` 是针对裸机 (bare-metal) 环境的交叉编译工具链，适用于编译运行在没有操作系统的 RISC-V 硬件上的程序。
>
> 但是，后面我们需要利用 BusyBox 构建根文件系统，而 BusyBox 需要 Linux 环境，所以我们选择安装 `riscv64-linux-gnu-gcc`.

因为笔者使用的是 Arch Linux, 可以直接使用包管理器 `pacman` 来安装，其他 Linux 发行版可以使用相应的包管理器，如 `apt` 等等：

```
sudo pacman -S riscv64-linux-gnu-gcc
```

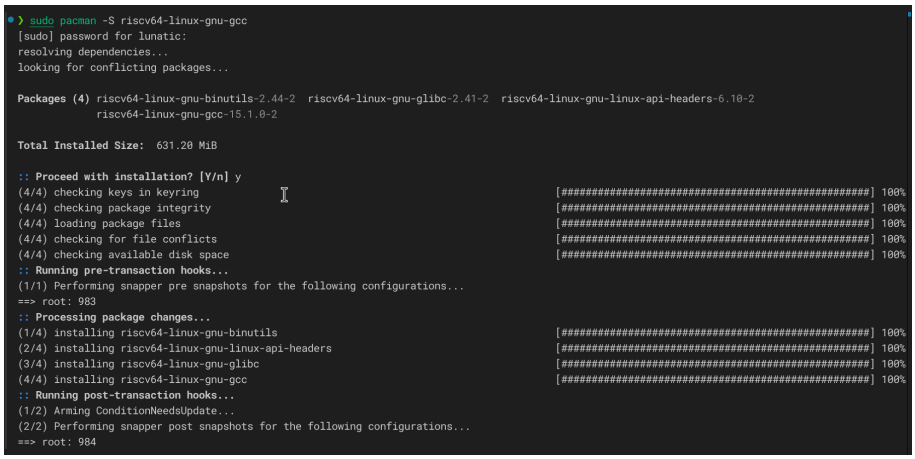


Figure 2: riscv64-linux-gnu-gcc 安装

安装完成后，运行下面的命令检查一下版本信息，确保安装成功：

```
riscv64-linux-gnu-gcc --version
```

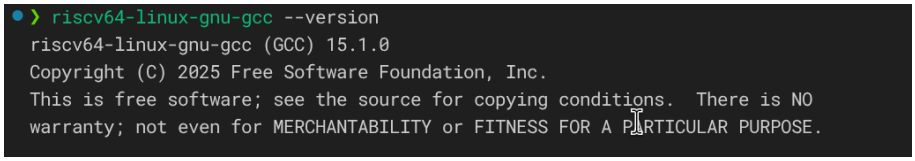


Figure 3: riscv64-linux-gnu-gcc 版本信息

**2. 下载并编译 Linux 内核**

这里，笔者下载的 Linux LTS 内核版本是 6.12.74, 可以从内核官方网站下载并解压：

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.12.74.tar.xz
tar -xJf linux-6.12.74.tar.xz
cd linux-6.12.74
```

运行 `make` 命令用默认的配置对内核进行配置，注意要选择 RISC-V 架构 (`ARCH=riscv`) 和交叉编译工具链 (`CROSS_COMPILE=riscv64-linux-gnu-`)：

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig
```

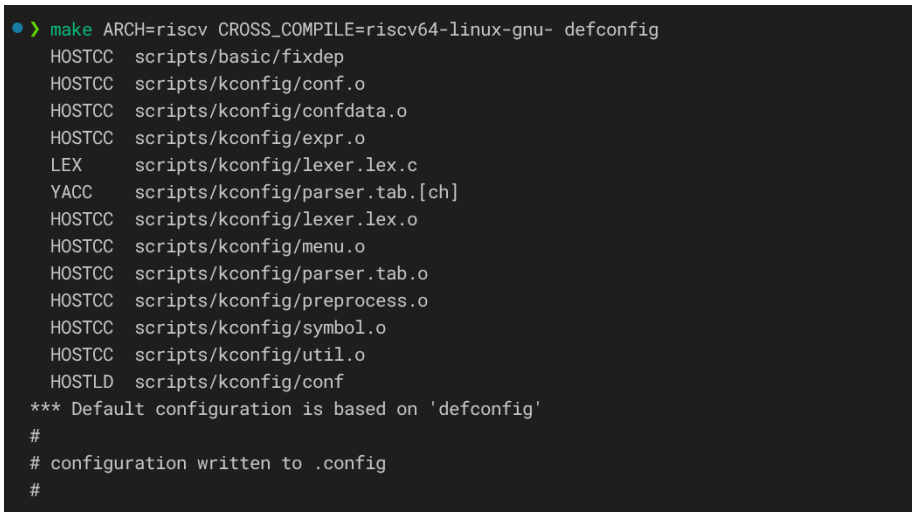


Figure 4: 内核配置

接着，需要对内核进行编译，在此之前，需要先安装一些系统命令，如 bc (可以用 sudo pacman -S bc 安装)，否则会在编译过程中报错．用下面的命令进行编译 Image:

```
make -j8 ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- Image
```

```
STUBCPY drivers/firmware/efi/libstub/secureboot.stub.o
STUBCPY drivers/firmware/efi/libstub/string.stub.o
STUBCPY drivers/firmware/efi/libstub/skip_spaces.stub.o
STUBCPY drivers/firmware/efi/libstub/systable.stub.o
STUBCPY drivers/firmware/efi/libstub/tpm.stub.o
STUBCPY drivers/firmware/efi/libstub/vsprintf.stub.o
STUBCPY drivers/firmware/efi/libstub/efi-stub-entry.stub.o
STUBCPY drivers/firmware/efi/libstub/riscv-stub.stub.o
STUBCPY drivers/firmware/efi/libstub/kaslr.stub.o
STUBCPY drivers/firmware/efi/libstub/riscv.stub.o
AR      drivers/firmware/efi/libstub/lib.a
AR      drivers/firmware/built-in.a
AR      drivers/built-in.a
AR      built-in.a
AR      vmlinux.a
LD      vmlinux.o
OBJCOPY modules.builtin.modinfo
GEN     modules.builtin
MODPOST vmlinux.symvers
CC      .vmlinux.export.o
UPD     include/generated/utsversion.h
CC      init/version-timestamp.o
KSYMS   .tmp_vmlinux0.kallsyms.S
AS      .tmp_vmlinux0.kallsyms.o
LD      .tmp_vmlinux1
NM      .tmp_vmlinux1.syms
KSYMS   .tmp_vmlinux1.kallsyms.S
AS      .tmp_vmlinux1.kallsyms.o
LD      .tmp_vmlinux2
NM      .tmp_vmlinux2.syms
KSYMS   .tmp_vmlinux2.kallsyms.S
AS      .tmp_vmlinux2.kallsyms.o
LD      vmlinux
NM      System.map
SORTTAB vmlinux
OBJCOPY arch/riscv/boot/Image
Kernel: arch/riscv/boot/Image is ready
```

Figure 5: 编译完成

编译完成后，运行 ls -lh 和 file 命令查看一下 Image 的大小和文件信息：

```
ls -lh arch/riscv/boot/Image
file arch/riscv/boot/Image
```

```
❯ ls -lh arch/riscv/boot/Image
-rwxr-xr-x 1 lunatic lunatic 24M Feb 25 09:52 arch/riscv/boot/Image
```

Figure 6: 内核大小

```
❯ file arch/riscv/boot/Image
arch/riscv/boot/Image: Linux kernel RISC-V boot executable Image, little-endian
```

Figure 7: 内核文件信息

至此，我们已经成功编译了 RISC-V Linux 内核，接下来我们需要构建根文件系统来运行这个内核.

**3. 编译 BusyBox**

我们使用 BusyBox 来构建根文件系统，首先下载 BusyBox 的源代码：

```
wget https://busybox.net/downloads/busybox-1.37.0.tar.bz2
tar -xjf busybox-1.37.0.tar.bz2
cd busybox-1.37.0
```

先生成默认配置：

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig
```

然后进入交互配置，将 `Settings -> Build Options -> Build static binary (no shared libs)` 选项勾选上：

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- menuconfig
```

> ⚠️ **ArchLinux 下进入 BusyBox 交互界面报错：`ncurses not found`**
>
> 笔者第一次配置时，提示需要安装 `ncurses-devel` 包，但用 `pacman` 安装了 `ncurses` 包后，问题依旧没有解决. 后来在 StackOverflow 上找到了答案.
>
> 解决这个问题，需要修改 `scripts/kconfig/lxdialog/check-lxdialog.sh` 文件，找到 `check()` 函数，在 `main()` 函数前面加上 `int`，改为 `int main()`，保存退出即可. 如下图所示
>
> ```
> 5 # Check if we can link to ncurses
> 4 check() {
> 3         $cc -x c - -o $tmp 2>/dev/null <<'EOF'
> 2 #include CURSES_LOC
> 1 int main() {} /int m        [1/1]
> 51 EOF
> ```
>
> Figure 8: 修改 `check-lxdialog.sh` 文件

进行编译：

```
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j8
```

```
❯ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- -j8
scripts/kconfig/conf -s Config.in
#
# using defaults found in .config
#
shell/hush.c: In function 'expand_one_var':
shell/hush.c:6642:15: warning: 'exp_word' is used uninitialized [-Wuninitialized]
 6642 |         char *exp_word = exp_word; /* for compiler */
      |               ^~~~~~~~
shell/hush.c:6642:15: note: 'exp_word' was declared here
 6642 |         char *exp_word = exp_word; /* for compiler */
      |               ^~~~~~~~
Static linking against glibc, can't use --gc-sections
Trying libraries: m resolv rt
 Library m is needed, can't exclude it (yet)
 Library resolv is needed, can't exclude it (yet)
 Library rt is not needed, excluding it
 Library m is needed, can't exclude it (yet)
 Library resolv is needed, can't exclude it (yet)
Final link with: m resolv
```

Figure 9: BusyBox 编译完成

**⚠️ ArchLinux 下 BusyBox 编译报错**

笔者一共遇到两个错误：一个是 `tc.c` 文件中 `CBQ` 相关的宏；另一个是 `sha1` 相关的编译报错.

`CBQ` 在 Linux 6.8 版本之后被移除了。好在其他的发行版已经有 patch 可以绕过 `CBQ` 了，执行以下命令修改 `networking/tc.c` 文件：

```
curl -L 'https://hina.lysator.liu.se/gentoo-portage/sys-apps/busybox/files/busybox-1.36.1-kernel-6.8.patch' \
| patch -p1
```

对于第二个问题，我们需要手动编辑 `libbb/hash_md5_sha.c` 文件，把 `shaNI` 的相关代码限制在 x86 架构下：

```
vim libbb/hash_md5_sha.c
```

然后执行修改：

```
1315    #if defined(__i386__) || defined(__x86_64__) /if defined(__i3
   1 // #if ENABLE_SHA1_HWACCEL
   2     || ctx->process_block == sha1_process_block64_shaNI
   3 #endif
```

Figure 10: 修改 `hash_md5_sha.c` 文件

编译完成后，运行 `file` 命令查看一下生成的 BusyBox 可执行文件：

```
❯ file busybox
busybox: ELF 64-bit LSB executable, UCB RISC-V, RVC, double-float ABI, version 1 (GNU/Linux), statically linked, BuildID[sha1]=d3c1f19980d073e02396
6ee85a48ff5bfd01f6e7, for GNU/Linux 4.15.0, stripped
```

Figure 11: 检查 BusyBox 可执行文件，发现的确是 statically linked 的

**4. 创建根文件系统**

我们先手动创建根文件系统的目录结构：

```
cd ..
export ROOTFS=$PWD/rootfs
mkdir -p $ROOTFS

mkdir -p "${ROOTFS}"/{bin,sbin,etc,proc,sys,dev,tmp,usr/{bin,sbin},var,root}
mkdir -p "${ROOTFS}/etc/init.d"
chmod 1777 "${ROOTFS}/tmp"
```

接着把 busybox 安装到刚刚创建的根文件系统中：

```
cd busybox-1.37.0
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- CONFIG_PREFIX="${ROOTFS}" install
```

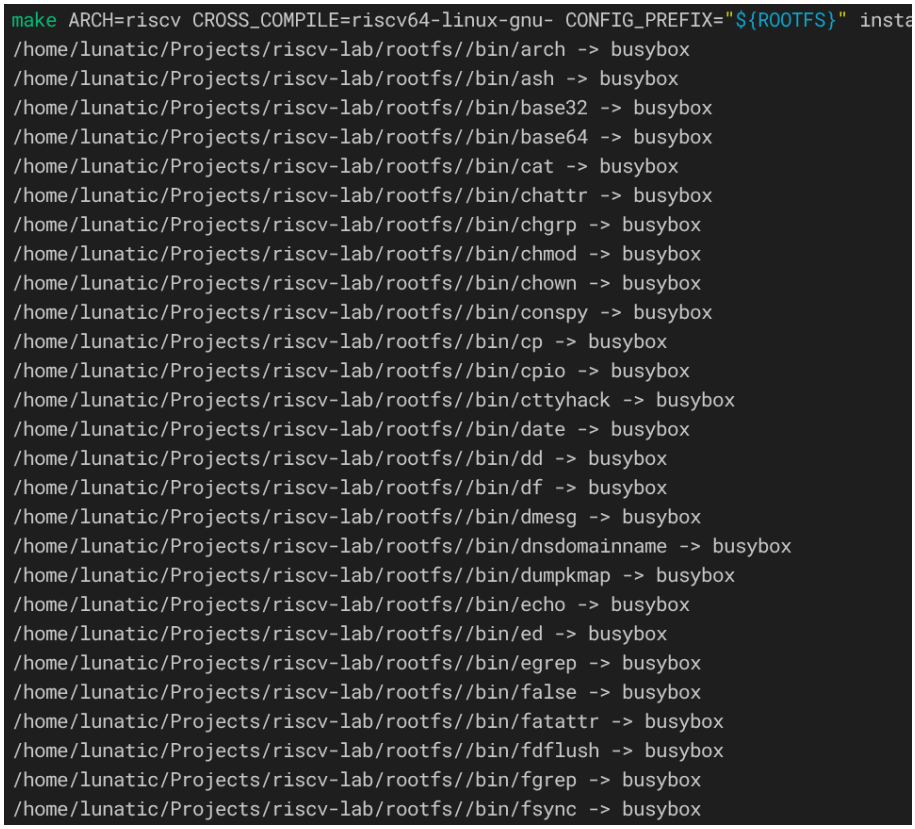


Figure 12: 安装 BusyBox 到 RootFS, 过程中会创建大量链接

接着创建 /dev 目录下的设备节点，至少要有 console，否则很容易没有输出或无法交互：

```
sudo mknod -m 600 "${ROOTFS}/dev/console" c 5 1
sudo mknod -m 666 "${ROOTFS}/dev/null" c 1 3
sudo mknod -m 666 "${ROOTFS}/dev/tty" c 5 0
sudo mknod -m 666 "${ROOTFS}/dev/zero" c 1 5
```

**5. 编写初始化脚本**

我们需要编写一个简单的初始化脚本来启动系统并自动挂载 /proc 和 /sys，创建一个 /etc/init.d/rcS 文件，命令如下：

```
cat > "${ROOTFS}/etc/init.d/rcS" <<'EOF'
#!/bin/sh

mount -t proc  proc /proc
mount -t sysfs sys  /sys

echo "Welcome, Arch Lunar."

exec /bin/sh
EOF

chmod +x "${ROOTFS}/etc/init.d/rcS"
```

对 initramfs, 内核默认会尝试执行根目录下的 /init. 所以我们用 /init 去调用 /etc/init.d/rcS

```
cat > "${ROOTFS}/init" <<'EOF'
#!/bin/sh
exec /etc/init.d/rcS
EOF

chmod +x "${ROOTFS}/init"
```

**6. 打包成 cpio.gz 格式**

我们用 fakeroot 和 cpio 命令把根文件系统打包成 cpio.gz 格式，注意可能需要先安装 fakeroot 和 cpio 包:

```
fakeroot -- sh -c 'find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initramfs.cpio.gz'
```

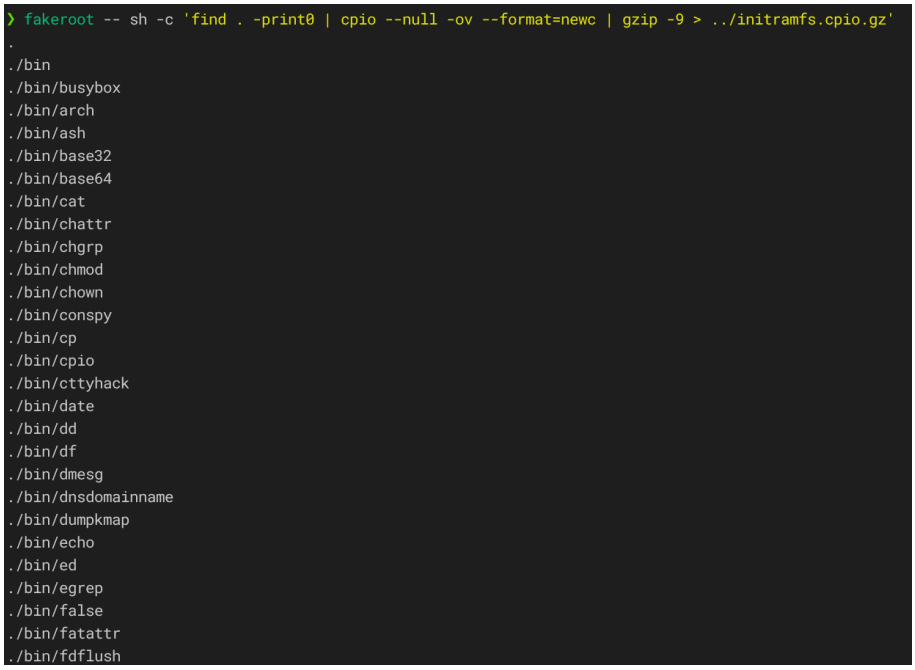


Figure 13: 打包成 cpio.gz 格式

Figure 14: 检查生成的 initramfs.cpio.gz 文件

**7. 编写 run.sh 脚本一键运行**

我们编写一个 run.sh 脚本来一键运行 qemu，命令如下：

```
#!/usr/bin/env bash
set -euo pipefail

KERNEL_IMAGE="${KERNEL_IMAGE:-./linux-6.12.74/arch/riscv/boot/Image}"
INITRAMFS="${INITRAMFS:-./initramfs.cpio.gz}"

# Checks
if [[ ! -f "$KERNEL_IMAGE" ]]; then
  echo "ERROR: kernel Image not found: $KERNEL_IMAGE" >&2
  exit 1
fi
if [[ ! -f "$INITRAMFS" ]]; then
  echo "ERROR: initramfs not found: $INITRAMFS" >&2
  exit 1
fi

# run qemu
exec qemu-system-riscv64 \
  -machine virt \
  -m 256M \
  -nographic \
  -kernel "$KERNEL_IMAGE" \
  -initrd "$INITRAMFS" \
  -append "console=ttyS0 earlycon=sbi rdinit=/init"
```

- -nographic 参数表示不使用图形界面，直接在终端中显示输出；
- -append 参数用来传递内核启动参数，这里我们指定了 console=ttyS0 来将控制台输出重定向到串口，earlycon=sbi 来启用早期控制台支持，以及 rdinit=/init 来指定内核启动后执行的初始化程序；
- -kernel 和 -initrd 参数分别指定了内核镜像和初始 RAM 文件系统。

> **? -nographic 的作用**
>
> 在 virt machine 下，qemu 默认会提供一个图形接口，同时仿真串口设备．如果不加 -nographic 参数，qemu 会在一个新的窗口中显示图形界面，而串口输出会被重定向到这个窗口中．如果加上 -nographic 参数，qemu 就不会启动图形界面，而是直接在当前终端中显示串口输出，这样我们就可以直接在终端中看到内核的启动日志和交互式 shell 的输出了．

## 8. 最终效果

```
❯ ./run.sh

OpenSBI v1.7
   ____                    _____ ____ _____
  / __ \                  / ____|  _ \_   _|
 | |  | |_ __   ___ _ __ | (___ | |_) || |
 | |  | | '_ \ / _ \ '_ \ \___ \|  _ < | |
 | |__| | |_) |  __/ | | |____) | |_) || |_
  \____/| .__/ \___|_| |_|_____/|____/_____|
        | |
        |_|

Platform Name               : riscv-virtio,qemu
Platform Features           : medeleg
Platform HART Count         : 1
Platform IPI Device         : aclint-mswi
Platform Timer Device       : aclint-mtimer @ 10000000Hz
Platform Console Device     : uart8250
Platform HSM Device         : ---
Platform PMU Device         : ---
Platform Reboot Device      : syscon-reboot
Platform Shutdown Device    : syscon-poweroff
Platform Suspend Device     : ---
Platform CPPC Device        : ---
Firmware Base               : 0x80000000
Firmware Size               : 317 KB
Firmware RW Offset          : 0x40000
Firmware RW Size            : 61 KB
Firmware Heap Offset        : 0x46000
Firmware Heap Size          : 37 KB (total), 2 KB (reserved), 11 KB (used), 23 KB (free)
Firmware Scratch Size       : 4096 B (total), 1400 B (used), 2696 B (free)
Runtime SBI Version         : 3.0
Standard SBI Extensions     : time,rfnc,ipi,base,hsm,srst,pmu,dbcn,fwft,legacy,dbtr,sse
Experimental SBI Extensions : none
```

Figure 15: 运行截图

```
[    0.000000] Linux version 6.12.74 (lunatic@arca) (riscv64-linux-gnu-gcc (GCC) 15.1.0, GNU ld (GNU Binutils) 2.44) #1 SMP Wed Feb 25 09:52:3
 2026
[    0.000000] random: crng init done
[    0.000000] Machine model: riscv-virtio,qemu
[    0.000000] SBI specification v3.0 detected
[    0.000000] SBI implementation ID=0x1 Version=0x10007
[    0.000000] SBI TIME extension detected
[    0.000000] SBI IPI extension detected
[    0.000000] SBI RFENCE extension detected
[    0.000000] SBI SRST extension detected
[    0.000000] SBI DBCN extension detected
[    0.000000] earlycon: sbi0 at I/O port 0x0 (options '')
[    0.000000] printk: legacy bootconsole [sbi0] enabled
[    0.000000] efi: UEFI not found.
[    0.000000] OF: reserved mem: 0x0000000080000000..0x000000008003ffff (256 KiB) nomap non-reusable mmode_resv1@80000000
[    0.000000] OF: reserved mem: 0x0000000080040000..0x000000008004ffff (64 KiB) nomap non-reusable mmode_resv0@80040000
[    0.000000] Zone ranges:
[    0.000000]   DMA32    [mem 0x0000000080000000-0x000000008fffffff]
[    0.000000]   Normal   empty
[    0.000000] Movable zone start for each node
[    0.000000] Early memory node ranges
[    0.000000]   node   0: [mem 0x0000000080000000-0x000000008004ffff]
[    0.000000]   node   0: [mem 0x0000000080050000-0x000000008fffffff]
[    0.000000] Initmem setup node 0 [mem 0x0000000080000000-0x000000008fffffff]
[    0.000000] SBI HSM extension detected
[    0.000000] riscv: base ISA extensions acdfhim
[    0.000000] riscv: ELF capabilities acdfim
[    0.000000] percpu: Embedded 22 pages/cpu s49640 r8192 d32280 u90112
[    0.000000] Kernel command line: console=ttyS0 earlycon=sbi rdinit=/init
[    0.000000] Dentry cache hash table entries: 32768 (order: 6, 262144 bytes, linear)
[    0.000000] Inode-cache hash table entries: 16384 (order: 5, 131072 bytes, linear)
[    0.000000] Built 1 zonelists, mobility grouping on.  Total pages: 65536
```

Figure 16: 滚动

```
[    0.394857] printk: legacy bootconsole [sbi0] disabled
[    0.417402] loop: module loaded
[    0.423520] e1000e: Intel(R) PRO/1000 Network Driver
[    0.423699] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[    0.425769] usbcore: registered new interface driver uas
[    0.426063] usbcore: registered new interface driver usb-storage
[    0.426959] mousedev: PS/2 mouse device common for all mice
[    0.431492] goldfish_rtc 101000.rtc: registered as rtc0
[    0.432127] goldfish_rtc 101000.rtc: setting system clock to 2026-02-25T02:43:16 UTC (1771987396)
[    0.435922] sdhci: Secure Digital Host Controller Interface driver
[    0.436079] sdhci: Copyright(c) Pierre Ossman
[    0.436330] Synopsys Designware Multimedia Card Interface Driver
[    0.436792] sdhci-pltfm: SDHCI platform and OF driver helper
[    0.437673] usbcore: registered new interface driver usbhid
[    0.437852] usbhid: USB HID core driver
[    0.438498] riscv-pmu-sbi: SBI PMU extension is available
[    0.439154] riscv-pmu-sbi: 16 firmware and 18 hardware counters
[    0.439382] riscv-pmu-sbi: Perf sampling/filtering is not supported as sscof extension is not available
[    0.445932] NET: Registered PF_INET6 protocol family
[    0.452046] Segment Routing with IPv6
[    0.452573] In-situ OAM (IOAM) with IPv6
[    0.453183] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[    0.456361] NET: Registered PF_PACKET protocol family
[    0.457593] 9pnet: Installing 9P2000 support
[    0.457966] Key type dns_resolver registered
[    0.489515] debug_vm_pgtable: [debug_vm_pgtable         ]: Validating architecture page table helpers
[    0.496893] clk: Disabling unused clocks
[    0.497170] PM: genpd: Disabling unused power domains
[    0.497356] ALSA device list:
[    0.497458]   No soundcards found.
[    0.540416] Freeing unused kernel image (initmem) memory: 2280K
[    0.541117] Run /init as init process
Welcome, Arch Lunar.
/bin/sh: can't access tty; job control turned off
~ #
```

Figure 17: Welcome 信息 + 终端提示符

```
~ # ls
bin      etc      linuxrc  root     sys      usr
dev      init     proc     sbin     tmp      var
```

Figure 18: `ls` 命令成功执行

```
~ # uname -a
Linux (none) 6.12.74 #1 SMP Wed Feb 25 09:52:38 CST 2026 riscv64 GNU/Linux
```

Figure 19: `uname -a` 命令成功执行

**❓ 为什么 x86 不能直接用 make 编译 RISC-V 内核?**

因为 x86 和 RISC-V 是两种不同的处理器架构，它们的指令集、寄存器结构、内存管理等方面都有很大的差异．直接 make 默认使用宿主机的工具链来生成宿主机平台的二进制文件，所以我们不能直接在 x86 上编译 RISC-V 内核，而需要使用交叉编译工具链来生成适用于 RISC-V 架构的二进制文件．

**🔥 宿主机与目标机的关系**

在交叉编译中，宿主机 (host) 是指我们进行编译的计算机系统，而目标机 (target) 是指我们希望运行编译结果的计算机系统．在这个实验中，宿主机是运行 Arch Linux 的 x86_64 计算机，而目标机是我们通过 qemu 模拟的 RISC-V 虚拟机．交叉编译工具链就是为了在宿主机上生成适用于目标机的二进制文件而设计的．

**? BusyBox 里输入 `ls` 回车，最终触发内核哪个机制读取目录**

从用户态开始

1. shell 首先解析命令，`fork()` 出一个子进程来执行 BusyBox `ls` applet
2. `execve()` 用 `openat()` 打开当前目录，获取目录的 `fd`，然后通过 `getdents64` 系统调用和 **VFS 目录迭代机制**把目录项从内核的文件系统实现 (ext4、tmpfs、ramfs 等) 拷贝到用户态。